

3D MO 액션 게임을 위한 glTF 2.0 표준 파서 및 스킨링 애니메이션 구현과 PBR 파이프라인 및 오픈소스 물리엔진 서버 최적화 연구

박대원, 임찬진, 박경준, 정내훈
한국공학대학교 게임공학과

june25656@tukorea.ac.kr ckswlsdl0916@gmail.com pkj7323@tukorea.ac.kr nhjung@tukorea.ac.kr

A Study on the Implementation of glTF 2.0 Standard Parser, Skinning Animation, PBR Pipeline, and Open-Source Physics Engine Server Optimization for 3D MO Action Games

Dae-Won Park, Chan-Jin Im, Kyung-Jun Park, Nai-Hoon Jung
Department of Game & Multimedia Engineering, Tech University Of Korea

요 약

본 연구에서는 3D 그래픽스 표준 포맷인 glTF 2.0 명세를 심층 분석하여, 외부 라이브러리에 의존하지 않는 독자적인 파서를 개발하고 스킨링 애니메이션 시스템을 성공적으로 구현하였다. 이를 바탕으로 DirectX 12 환경에서 실시간 물리 기반 렌더링(PBR) 파이프라인을 구축하여 클라이언트 렌더링 성능과 시각적 사실성을 확보하였다. 또한, 3D MO 액션 게임 환경에 맞춰 IOCP 기반 비동기 네트워크 서버에 Jolt Physics 엔진을 연동하여 서버 권위적(Server-Authoritative) 물리 시뮬레이션 환경을 완성하였다. 이를 통해 스냅샷 리와인드 기반의 지연 보상 시스템과 물리 LOD를 적용하여 다중 접속 시 발생하는 물리 연산의 병목 현상을 해소하고 네트워크 동기화의 신뢰성을 확보하였다.

1. 서 론

1.1 연구 배경 및 목적

현대의 3D 액션 게임 개발에 있어 정교한 물리 연산과 수준 높은 그래픽스 파이프라인은 필수적이다. 그러나 범용 3D 에셋 로딩 라이브러리(Assimp 등)나 상용 엔진에 의존할 경우, 불필요한 오버헤드로 인해 로딩 성능이 저하되고 DirectX 12 환경에 맞춘 세밀한 메모리 최적화에 한계가 따른다.

이에 본 연구에서는 다음과 같은 세 가지 핵심 기술을 통합한 자체 엔진 및 서버 아키텍처를 개발하였다. 첫째, 3D 모델 데이터를 직접 파싱하여 에셋 로딩 성능을 극대화하였다. 둘째, 멀티 스캐터링 보정을 적용하여 에너지 보존 법칙을 준수하는 물리적으로 정확한 렌더링(PBR) 환경을 구축하였다. 셋째, 멀티스레드 환경에서 오픈소스 물리 엔진의 병목을 최소화하고, 서버 권위적 아키텍처를 도입하여 클라이언트 변조를 원천 차단하는 신뢰성 높은 MO(Multiplayer Online) 액션 게임 서버를 완성하였다.

1.2 시스템 주요 특징

독자적 렌더링 파이프라인: 이진 탐색(Binary Search) 기반의 애니메이션 키프레임 탐색과 Linear Allocator를 활용한 다중 프레임 메모리 할당 최적화. 물리적 정확성 확보: 멀티 스캐터링 보정이 적용된 Cook-Torrance BRDF 및 Split-Sum Approximation 기반의 이미지 기반 라이팅(IBL) 통합. 서버 주도적 아키텍처: 인스턴스(Room) 단위의 물리 공간 격리, 브로드페이즈(Broad-Phase) 충돌 컬링, 스냅샷 기반 지연 보상 시스템 구축.

1.3 개발 플랫폼 및 장르

플랫폼: Windows PC (DirectX 12, IOCP)
장르: 3D Multiplayer Online Action RPG

2. 연구 내용

2.1 glTF 표준 명세 분석 및 독자적 파싱 모듈 구현

범용 3D 로드 라이브러리(Assimp 등)가 유발하는 방대한 범용 트리(Tree) 자료구조 생성 및 포인터 추적(Pointer Chasing) 오버헤드를 제거하기 위해 독자적인 glTF 파싱 시스템을 구현하였다. 바이너리

파일(.bin)을 로드하여, glTF 명세에 맞춰 바이너리 파일로부터 정점 좌표, 노멀, UV, 탄젠트 및 스키닝 데이터를 바이트 스트라이드 기반으로 추출한 직후, 이를 1차원 구조체 배열로 조립하여 메모리 캐시 효율을 극대화하였다.[1] 또한, 오른손 좌표계를 DirectX 12의 왼손 좌표계에 맞추기 위해 Z축을 반전시키는 스케일 행렬 $S(1, 1, -1)$ 을 역바인드 행렬 양단에 곱하여 이기종 좌표계 변환을 처리하였다.

2.1.1 뼈대 행렬 보간 및 키프레임 탐색 알고리즘 최적화

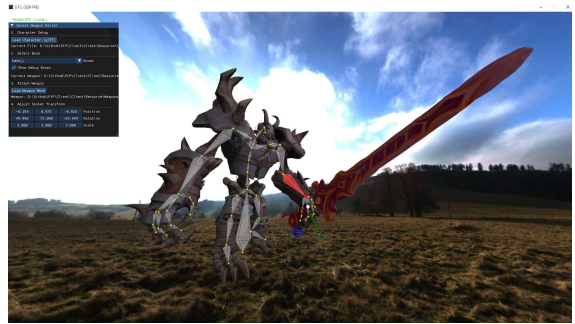
다양한 보간 방식(Linear, Step, Cubic Spline)을 분석해 시간 변화율(t)에 따른 이동, 회전, 크기 값을 프레임마다 연산한다.[1] 특히 정렬된 시간 배열의 특성을 활용해 std::upper_bound 기반의 이진 탐색을 도입함으로써, 기존 선형 탐색의 $O(N)$ 비용을 $O(\log N)$ 으로 단축시켜 CPU 연산 효율을 높였다.

2.1.2 마스터 스켈레톤(Master Skeleton) 및 소켓 시스템(Socket System)

분리된 캐릭터 파츠(투구, 갑옷 등)를 렌더링하기 위해 0번 뼈대를 마스터 스켈레톤으로 지정하고, 추가 파츠가 이를 참조하는 Joint Index Mapping 구조를 설계해 렌더링 뼈대 일관성을 확보했다. 또한, 행렬 곱셈 연산을 기반으로 특정 뼈대에 무기 등의 객체를 부착하고 애니메이션과 동기화하는 소켓 기능을 구현하였다.



[그림1] 마스터 스켈레톤이 적용된 통합 메쉬 그림



[그림2] 소켓 기능을 통해 특정 뼈대에 객체를 부착한 그림

2.1.3 다중 프레임 렌더링을 위한 선형 할당 기반 메모리 최적화

다중 프레임 렌더링 시 발생하는 상수 버퍼 동적 할당 오버헤드를 막고자 선형 할당기를 직접 설계하였다. 거대한 업로드 힙을 초기 1회만 할당하고, DirectX 12의 256바이트 정렬(CB_ALIGNMENT) 규칙에 맞춰 프레임별 메모리 블록을 분할했다.[2] 매 프레임 할당기 오프셋만 순차 증가시켜 $O(1)$ 비용으로 즉시 메모리를 활용해 단편화를 원천 차단하였다.

2.1.4 계층적 OBB 및 BVH 구조를 활용한 카메라 충돌 연산 최적화

거대 메쉬와 카메라 간의 실시간 충돌을 최적화하기 위해 계층적 OBB 연산을 도입하였다. 세부 프리미티브별 바운딩 박스를 포괄하는 '마스터 OBB' 기반의 BVH를 구축하여, 마스터 OBB와 우선 교차 판별 후 충돌 시에만 하위 OBB를 검사하는 2단계 구조를 채택함으로써 연산 오버헤드를 대폭 감소시켰다.

2.1.5 무게중심 좌표계를 활용한 표면 파티클 추출

메쉬 표면에서 균일한 파티클을 생성하기 위해 삼각형 면적 기반의 가중치 난수 알고리즘을 도입하였다. 무게중심 좌표계(Barycentric Coordinates)를 적용하여 정점 내부에 뭉침 없는 균일한 타겟 좌표를 생성하는 로직을 구현하였다.[3] 이를 통해 파티클 이펙트가 타겟 메쉬의 형태를 서서히 갖춰가도록 처리하였다.



[그림3] 파티클들이 서서히 메쉬의 형태를 갖추어나가는 그림



[그림4] 파티클들이 지정된 메쉬 표면으로 모여 메쉬의 형태를 갖춘 그림

2.2 실시간 PBR 파이프라인 및 glTF 2.0 기반 IBL 렌더링 구현

본 연구에서는 glTF 2.0의 Metallic-Roughness 워크플로우를 기반으로, 물리 법칙에 맞게 빛을 연산하는 실시간 PBR(Physically Based Rendering) 파이프라인을 구축했다. 또한, 물체가 주변 환경의 빛을 자연스럽게 반사하고 흡수하도록 만드는 이미지 기반 조명(IBL, Image-Based Lighting) 기술을 통합하여 시각적 사실성을 극대화했다.

2.2.1 glTF 2.0 표준 기반의 텍스처 데이터 해석

효율적인 렌더링을 위해 하나로 압축된 텍스처 데이터를 엔진이 물리적 수치로 해석하는 과정을 구현했다. 메모리 대역폭 절약을 위해 한 장의 이미지에 합쳐진 ORM 텍스처를 활용하며, 셰이더 단계에서 이를 분리하여 R 채널은 차폐(Occlusion), G 채널은 표면의 거칠기(Roughness), B 채널은 금속성(Metallic)으로 해석한다. 텍스처에서 샘플링한

각 데이터에는 상수 버퍼로 전달된 재질 고유의 glTF Factor 값을 곱하여 최종적인 PBR 파라미터를 결정한다.

2.2.2 직접광(Direct Lighting) 연산을 위한

미세면(Microfacet) BRDF 모델

결정된 PBR 파라미터를 바탕으로, 직접광에 의한 표면 반사는 물리적 타당성을 확보하기 위해 Cook-Torrance BRDF 모델을 기반으로 계산된다. 법선 분포 함수(NDF)로는 GGX 모델을 사용하여 기존 모델보다 하이라이트의 꼬리가 길게 퍼지는 현실적인 금속 질감을 표현했다. 복잡한 연산을 최소화하기 위해 아래와 같이 최적화된 공식을 적용했다.

$$NDF = a2 / (\pi \times ((NoH \times a2 - NoH) \times NoH + 1)^2)$$

(단, a2는 Roughness의 제곱, NoH는 법선과 하프벡터의 내적)

기하 감쇠 함수(Geometry)는 미세한 요철에 의해 빛이 가려지거나 막히는 현상을 효율적으로 계산하기 위해 Smith Joint 근사법을 적용했다. 빛의 방향과 시선 방향의 가려짐을 한 번에 계산하여 연산량을 감축했다.

$$Visibility =$$

$$0.5$$

$$NoL \times (NoV \times (1 - a) + a) + NoV \times (NoL \times (1 - a) + a)$$

(단, a는 Roughness의 제곱, NoL과 NoV는 각각 법선과 빛, 법선과 시선의 내적)

또한, 프레넬 방정식(Fresnel)은 Schlick 근사식을 사용하여 시선 각도에 따른 반사율 변화를 구현하였으며, 비금속 물질의 기본 반사율(F0)을 4%로 고정하였다.[4, 5]

$$Fresnel = F0 + (1.0 - F0) \times (1.0 - VoH)^5$$

(단, VoH는시선과하프벡터의내적)

2.2.3 실시간 환경광 처리를 위한 IBL(Image-Based Lighting) 기법 도입

직접광만으로는 표현할 수 없는 간접광 및 주변 환경의 반사를 실시간으로 렌더링하기 위해 IBL 기법을 적용했다. 빛이 물체 표면 아래로 산란되어 흩어지는 난반사광(Diffuse IBL)은 무거운 환경맵을 직접 샘플링하는 대신, 미리 계산된 9개의 구면 조화(SH) 계수를 사용하여 연산량을 획기적으로 감축하면서도 부드러운 주변 환경광을 재현한다.[6]

$$DiffuseIBL = k_D \times Calculate.SH(Normal) \times Albedo$$

(단, kD는 에너지 보존을 위한 난반사율, CalculateSH는 9개의 구면 조화 계수와 법선 벡터의 다항식 연산)

이를 통해 거친 표면에 주변 환경이 흐릿하게 맺히거나 선명하게 반사되는 정반사광(Specular IBL) 현상을 물리적으로 타당하게 보여준다.

2.2.4 실시간 렌더링 성능 확보를 위한 Split-Sum Approximation

주변 환경을 전방위로 샘플링해야 하는 정반사 IBL 렌더링 방식의 막대한 연산 비용을 해결하기 위해, Epic Games가 제안한 분할 합 근사(Split-Sum Approximation) 방식을 적용했다.[5] 이 기법은 복잡한 렌더링 적분식을 두 개의 개별적인 사전 계산(Pre-computed) 데이터로 분리하여 성능을 확보한다. 첫 번째로, 표면의 거칠기에 따라 환경이 흐릿하게 반사되는 현상을 보여주기 위해 Roughness 수치를 Mipmap 레벨에 매핑하여 사전 필터링된 환경맵을 샘플링한다. 두 번째로, 거칠기와 시선 각도(NdotV)를 각각 2D 텍스처의 축으로 삼아 환경광 반사에 필요한 스케일 및 바이어스 값을 사전 계산한 BRDF LUT를 사용한다.[4, 7] 픽셀 셰이더 단계에서는 이 두 가지 결과값을 조합하여 최종적인 정반사 환경광을 실시간으로 도출한다.

$SpecularIBL = PrefilteredColor \times (F0 \times BRDF_{LUT,x} + BRDF_{LUT,y})$
(단, PrefilteredColor는 환경맵 샘플링 결과, BRDF_LUT.x와 y는 각각 사전 계산된 Scale과 Bias 값)

2.2.5 PBR 렌더링 파이프라인의 시각적 검증

구현된 PBR 파이프라인의 물리적 타당성을 검증하기 위해 Khronos Group의 'Damaged Helmet' 샘플 모델을 활용하였다.[8] 이 모델은 단일 에셋 내에 0.0부터 1.0 범위의 다양한 Metallic 및 Roughness 수치가 복합적으로 매핑되어 있어 렌더링 알고리즘을 평가하기에 적합하다.



[그림5] 자체 엔진에서 렌더링한 Khronos Group의 'Damaged Helmet' 샘플 모델, Metallic 수치에 따른 정확한 반사율 적용과 Roughness 수치 변화에 따른 환경광 산란 및 정반사 차폐 현상이 적용됨

2.3 멀티플레이어 서버 물리엔진 통합 및 최적화

2.3.1 물리공간 격리 및 물리객체 필터링

서버의 각 Room 인스턴스마다 독립적인 물리 공간을 할당하고, 로직 스레드별로 TempAllocator를 재사용하도록 설계하여 동적 메모리 할당 오버헤드를 원천적으로 제거하였다. 또한, 대규모 NPC 운용 시 발생하는 연산 부하를 완화하기 위해 계층적 충돌 필터링(Hierarchical Collision Filtering)을 도입하였다. 이를 통해 Broad-phase 단계에서 불필요한 충돌 쌍을 조기에 차단함으로써 전체적인 물리 시뮬레이션 효율을 극대화하였다.[8, 9]

2.3.2 정적 Map Data 메모리 공유 및 인스턴싱 최적화

모든 Room 인스턴스가 공유하는 정적 물리 객체의 중복 생성을 막기 위해, 서버 초기화 단계에서 물리 형태(Shape) 데이터를 단일 생성하여 메모리에 적재하였다. 특히 glTF 기반의 MeshShape[1]와 단순 충돌체(Box, Capsule 등)를 결합한 StaticCompoundShape 기능을 사용 및 구현하여 정적 물리 객체의 충돌 오버헤드를 감소시켰다. 각 인스턴스 생성 시에는 기 적재된 메모리 주소를 참조하여 물리 객체(Body)를 인스턴싱함으로써 서버 가용 메모리 공간을 효율적으로 확보하였다. [1, 8, 9]

2.3.3 물리 공간 분할 및 거리 기반 물리 LOD 적용

대규모 NPC 운용 시 발생하는 CPU 병목을 해소하고자 플레이어 시야 기반의 공간 분할(GridMap) 물리 LOD를 적용하였다. 플레이어 주변의 주요 객체는 Kinematic 타입으로 설정하여 정밀한 물리 업데이트를 수행하는 반면, 원거리 객체는 RayCast 기반의 위치 제어로 전환하고 메인 업데이트 루프(update 함수) 호출을 제한함으로써 성능을 최적화하였다. 또한 계층적 충돌 필터링을 통해 Broad-phase 단계에서 불필요한 충돌 쌍을 조기에 차단하여 시뮬레이션 효율을 극대화하였다. 제한된 서버 리소스(AMD Ryzen 5 5600X, 로직 스레드 9개)와 대규모 밀집 부하 조건(100개 Room, 총 400명 플레이어 및 50,000기 NPC)에서 본 최적화 기법을 적용하여 측정된 스레드 점유율과 평균 틱 지연(Tick Delay) 결과는 각각 [그림 6],[그림 7]과

같으며 빨간색 막대 그래프가 최적화 전, 초록색 막대 그래프가 최적화 후를 나타낸다.



[그림 6] 물리 LOD 적용에 따른 스레드 점유율 비교



[그림 7] 물리 LOD 적용에 따른 평균 틱 지연 비교

2.3.4 계층적 컴포넌트 구조 기반의 객체 물리 제어 시스템

NPC의 효율적인 로직 처리를 위해 기초 데이터 관리부터 AI 이동 제어에 이르는 계층적 컴포넌트 시스템을 구축하였다. 본 시스템은 엔티티의 위치와 회전을 담당하는 TransformComponent와 물리 엔진의 Low-Level API를 추상화한 CharacterControllerComponent를 제작했다. 이것을 기반으로 그 상위에서 AI 엔진의 이동 벡터를 물리 컨트롤러에 전달하는 NPCControllerComponent가 동작하도록 구현했으며, 최상위 NPC 클래스가 이들을 소유하여 생명주기를 관리한다. 정밀 타격 판정을 위한 HitboxComponent 또한 아키텍처에 포함되어 있으나, 네트워크 지연 보상을 위한 상세 세부 로직은 후술할 지연 보상 시스템(2.3.5)과 연계하여 처리하도록 설계하였다.

2.3.5 스냅샷 리와인드 기반 지연 보상 및 부위별 타격 시스템

네트워크 지연환경에서도 공정한 타격 판정을 보장하기 위해 스냅샷 리와인드 기반의 지연 보상(Lag Compensation) 아키텍처를 도입하였다. 앞서 언급한 HitboxComponent는 캐릭터의 진신 충돌체와 별개로 머리, 몸통, 팔다리 등 세부 부위별 형상(HitboxData)을 관리한다. 공격 패킷 수신 시 서버는 타겟의 과거 트랜스폼 데이터를 역산하여 위치를 복원하며, 물리엔진에 구현되어 있는 sCollideShapeVsShape 함수를 통해 공격 형상과 세부 히트박스 간의 1:1 교차 검증을 수행한다. 이를 통해 핑(Ping) 차이에 관계없이 정확한 부위별 피격 판정과 그에 따른 차등 데미지 산출을 구현하였다.

3. 결 론

3.1 개발 요약

본 연구는 DirectX 12와 IOCP를 기반으로 렌더링, 애니메이션, 서버 네트워크, 물리 시뮬레이션에 이르는 전 과정을 독자적 수단으로 통합 설계하여 상용 엔진 수준의 MO 액션 서버 환경을 성공적으로 구축하였다.

3.2 장점

- 1) 고가용성 및 성능 향상: Linear Allocator와 전역 메모리 공유(ShapeRef)를 통해 다수의 객체 생성 시 메모리 단편화와 지연을 획기적으로 개선하였다.
- 2) 시각적 사실성과 렌더링 최적화: Split-Sum 근사법과 구면 조화 함수(SH)를 활용한 IBL 파이프라인을 구축하여, 막대한 환경광 연산 비용을 낮추면서도 물리적으로 자연스러운 빛 반사를 실시간으로 구현하였다.
- 3) 무결성 및 신뢰성: 서버 권위적 판정과 스냅샷 기반 지연 보상 시스템을 결합하여, 클라이언트 변조를 원천 차단하고 네트워크 지연 상황에서도 높은 타격 신뢰성을 보장한다.

3.3 향후 개발 방향

GPU 기반 컴퓨터 세이더 도입: CPU 스키닝 연산을 Compute Shader로 오프로딩하여 병렬 처리 성능을 극대화할 계획이다. 애니메이션 및 AI 고도화: 상태 머신(FSM)과 가중치 기반 블렌딩을 도입하고, PPO(Proximal Policy Optimization) 기반 강화학습 AI를 결합하여 지능형 NPC를 설계할 예정이다. 동적 물리 확장: 물 유체(Fluid)나 천(Cloth) 시뮬레이션까지 서버 권위적으로 제어하는 환경을 연구하고자 한다.

4. 참고문헌

- [1] The Khronos Group, "glTF 2.0 Specification", Khronos Registry, 2017.
(URL:<https://registry.khronos.org/glTF/specs/2.0/glTF-2.0.html>)
- [2] Microsoft, "Upload and readback of texture data", Microsoft Learn, 2023.
(URL:<https://learn.microsoft.com/ko-kr/window/s/win32/direct3d12/upload-and-readback-of-te>)

- [xture-data](#))
- [3] Matt Pharr, Wenzel Jakob, and Greg Humphreys, "Sampling a Triangle", Physically Based Rendering v3, 2018.
(URL: https://pbr-book.org/3ed-2018/Monte_Carlo_Integration/2D_Sampling_with_Multidimensional_Transformations#SamplingaTriangle)
 - [4] <https://github.com/google/filament/tree/main>
 - [5] Brian Karis (Epic Games).(2013).
"Real Shading in Unreal Engine 4".
 - [6] <https://github.com/dariomanesku/cmft/tree/c0ce2d34f90587f1d8a85cee9df1cf47fc428342>
ACM SIGGRAPH 2013 Courses.
 - [7] Joey de Vries (LearnOpenGL).
(URL: <https://learnopengl.com/PBR/IBL>)
 - [8] <https://github.com/KhronosGroup/glTF-Sample-Models/tree/main>
 - [9] <https://github.com/jrouwe/JoltPhysics>
 - [10] <https://jrouwe.github.io/JoltPhysics>

본 연구는 2026년도 과학기술정보통신부 및 정보통신기획평가원의 'SW중심 대학사업' 지원을 받아 수행되었음(2025-0-00050)